

UNIT: 9

EVENT, HANDLING EVENTS AND SWING

9.1 Basic of event handling:

- An **event** in Java is an object that is created when something changes within a graphical user interface.
- If a user clicks on a button, clicks on a combo box, or types characters into a text field, etc., then an event triggers, creating the relevant event object.
- This behavior is part of Java's Event Handling mechanism and is included in the Swing GUI library.
- Events may also occur that are not directly caused by interactions with a user interface. For example, an event may be generated when a timer expires, a counter exceeds a value, software or hardware failure occurs, or an operation is completed.

9.2 Event Handling:

- Event handling is fundamental to Java programming because it is integral to the creation of applets and other types of GUI-based programs.
- Any program that uses a graphical user interface, such as a Java application written for Windows, is event driven.
- Thus, we cannot write these types of programs without a solid command of event handling.
- Events are supported by a number of packages, including ***java.util***, ***java.awt***, and ***java.awt.event***.
- Most events to which our program will respond are generated when the user interacts with a ***GUI-based program***.
- There are several types of events, including those generated by the ***mouse***, ***the keyboard***, and ***various GUI controls, such as a push button, scroll bar, or check box***.
- In all cases, the same basic event handling techniques apply.

❖ Event Delegation Model/Event Handling Model:[Imp for exam]

- The Delegation event model is one of the many techniques used to handle events in GUI (Graphical User Interface) programming languages.
- GUI represents a system where a user visually/graphically interacts with the system.
- In the *delegation* model, an event is an *object* that describes a state change in a source.
- Some of the activities that cause events to be generated are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse.
- This is the basis of event handling.

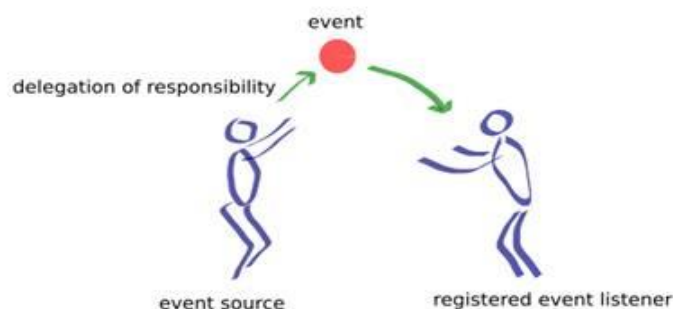


Figure: Events, sources, and listeners

There are three participants in event delegation model in Java as shown in above figure;

Event Source – the class which broadcasts the events

Event Listeners – the classes which receive notifications of events

Event Object – the class object which describes the event.

Event object:

- The event object defines the change in state in the event source class.
- For example, interacting with the graphical interfaces, such as clicking a button or entering text via keyboard in a text box, item selection in a list, etc.
- The event object is used to carry the required information about the state change. However, all events are not cause by user interaction.

Event sources:

- Event sources are objects that cause the events to occur due to some change in the property of the component.
- Because there can be various types a component can trigger, each must be registered to a listener to provide a suitable response.

Following table lists some of the user interface components that can generate the events.

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Menu Item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table: Sources of Events

Event listeners:

- Event listeners are objects that are notified as soon as a specific event occurs.
- Event listeners must define the methods to process the notification they are interested to receive.
- It has two major requirements. First, it must have been *registered with one or more sources to receive notifications about specific types of events*. Second, it *must implement methods to receive and process these notifications*.
- For example, the *MouseMotionListener* interface defines two methods to receive notifications when the mouse is dragged or moved.
- Any object may receive and process one or both of these events if it provides an implementation of this interface

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Table: Commonly Used Event Listener Interfaces

9.3. Event Classes:

- The classes that represent events are at the core of Java's event handling mechanism.
- Thus, a discussion of event handling must begin with the event classes.
- The most widely used events are those defined by the AWT and those defined by Swing.
- At the root of the Java event class hierarchy is `EventObject`, which is in `java.util`.
- `EventObject` is a superclass of all events.
- ***AWTEvent*** is a superclass of all AWT events that are handled by the delegation event model.
- The package `java.awt.event` defines many types of events that are generated by various user interface elements.

Following table shows several commonly used event classes and provides a brief description of when they are generated.

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Fig: Main Event Classes in `java.awt.event`

- **The ActionEvent Class:**

- ✓ An *ActionEvent* is generated when a *button* is pressed, a *list item* is double-clicked, or a *menu item* is selected.
- ✓ The *ActionEvent* class defines four integer constants that can be used to identify any modifiers associated with an action event: *ALT_MASK*, *CTRL_MASK*, *META_MASK*, and *SHIFT_MASK*.
- ✓ In addition, there is an integer constant, *ACTION_PERFORMED*, which can be used to identify action events.

ActionEvent has these three constructors:

ActionEvent(Object src, int type, String cmd)

ActionEvent(Object src, int type, String cmd, int modifiers)

ActionEvent(Object src, int type, String cmd, long when, int modifiers)

- ✓ For example, when a button is pressed, an action event is generated that has a command name equal to the label on that button.

- **The AdjustmentEvent Class:**

- ✓ An *AdjustmentEvent* is generated by a scroll bar.
- ✓ There are five types of adjustment events.
- ✓ The *AdjustmentEvent* class defines integer constants that can be used to identify them.

The constants and their meanings are shown here:

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

In addition, there is an integer constant, *ADJUSTMENT_VALUE_CHANGED*, that indicates that a change has occurred. Here is one *AdjustmentEvent* constructor:

***AdjustmentEvent*(Adjustable src, int id, int type, int data)**

For example, when a scroll bar is manipulated, this method returns the value represented by that change.

- **The ComponentEvent Class:**

- ✓ A *ComponentEvent* is generated when the *size*, *position*, or *visibility* of a component is changed.
- ✓ There are four types of component events.
- ✓ The *ComponentEvent* class defines integer constants that can be used to identify them.

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

ComponentEvent has this constructor:

- **The ContainerEvent Class:**

- ✓ A *ContainerEvent* is generated when a component is added to or removed from a container.
- ✓ There are two types of container events.
- ✓ The *ContainerEvent* class defines int constants that can be used to identify them: *COMPONENT_ADDED* and *COMPONENT_REMOVED*.
- ✓ They indicate that a component has been added to or removed from the container.

ContainerEvent is a subclass of *ComponentEvent* and has this constructor:

ContainerEvent(Component src, int type, Component comp)

- **The FocusEvent Class:**

- ✓ A *FocusEvent* is generated when a component gains or loses input focus.
- ✓ These events are identified by the integer constants *FOCUS_GAINED* and *FOCUS_LOST*.

FocusEvent is a subclass of *ComponentEvent* and has these constructors:

- ✓ *FocusEvent(Component src, int type)*
- ✓ *FocusEvent(Component src, int type, boolean temporaryFlag)*
- ✓ *FocusEvent(Component src, int type, boolean temporaryFlag, Component other)*
- ✓ If a *FOCUS_GAINED* event occurred, *other* will refer to the component that lost focus.
- ✓ Conversely, if a *FOCUS_LOST* event occurred, *other* will refer to the component that gains focus.

- **The InputEvent Class:**

- ✓ The abstract class *InputEvent* is a subclass of *ComponentEvent* and is the superclass for component input events.
- ✓ Its subclasses are *KeyEvent* and *MouseEvent*. *InputEvent* defines several integer constants that represent any modifiers, such as the control key being pressed, that might be associated with the event. Originally, the *InputEvent* class defined the following eight values to represent the modifiers:

ALT_DOWN_MASK	BUTTON2_DOWN_MASK	META_DOWN_MASK
ALT_GRAPH_DOWN_MASK	BUTTON3_DOWN_MASK	SHIFT_DOWN_MASK
BUTTON1_DOWN_MASK	CTRL_DOWN_MASK	

- **The ItemEvent Class:**

- ✓ An *ItemEvent* is generated when a *check box* or a *list item* is *clicked* or when a *checkable menu item* is *selected* or *deselected*.
- ✓ There are two types of item events, which are identified by the following integer constants:

DESELECTED	The user deselected an item.
SELECTED	The user selected an item.

ItemEvent(ItemSelectable src, int type, Object entry, int state)

Here, *src* is a reference to the component that generated this event. For example, this might be a *list* or *choice element*. The type of the event is specified by *type*. The specific item that generated the item event is passed in *entry*. The current state of that item is in *state*.

- **The KeyEvent Class:**

- ✓ A *KeyEvent* is generated when keyboard input occurs.
- ✓ There are three types of key events, which are identified by these integer constants: *KEY_PRESSED*, *KEY_RELEASED*, and *KEY_TYPED*.
- ✓ The first two events are generated when any key is pressed or released.
- ✓ The last event occurs only when a character is generated.

There are many other integer constants that are defined by *KeyEvent*.

For example, *VK_0* through *VK_9* and *VK_A* through *VK_Z* define the ASCII equivalents of the numbers and letters. Here are some others:

<i>VK_ALT</i>	<i>VK_DOWN</i>	<i>VK_LEFT</i>	<i>VK_RIGHT</i>
<i>VK_CANCEL</i>	<i>VK_ENTER</i>	<i>VK_PAGE_DOWN</i>	<i>VK_SHIFT</i>
<i>VK_CONTROL</i>	<i>VK_ESCAPE</i>	<i>VK_PAGE_UP</i>	<i>VK_UP</i>

The *VK* constants specify virtual key codes and are independent of any modifiers, such as *control*, *shift*, or *alt*. *KeyEvent* is a subclass of *InputEvent*. Here is one of its constructors:

KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)

- **The MouseEvent Class:(Imp for exam):**

- ✓ There are eight types of mouse events.
- ✓ The *MouseEvent* class defines the following integer constants that can be used to identify them:

<i>MOUSE_CLICKED</i>	The user clicked the mouse.
<i>MOUSE_DRAGGED</i>	The user dragged the mouse.
<i>MOUSE_ENTERED</i>	The mouse entered a component.
<i>MOUSE_EXITED</i>	The mouse exited from a component.
<i>MOUSE_MOVED</i>	The mouse moved.
<i>MOUSE_PRESSED</i>	The mouse was pressed.
<i>MOUSE_RELEASED</i>	The mouse was released.
<i>MOUSE_WHEEL</i>	The mouse wheel was moved.

MouseEvent is a subclass of *InputEvent*.

Here is one of its constructors:

MouseEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup)

Here, *src* is a reference to the component that generated the event. The type of the event is specified by *type*. The system time at which the mouse event occurred is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in *x* and *y*. The click count is passed in *clicks*.

It returns a value that represents the button that caused the event. The return value will be one of these constants defined by *MouseEvent*:

<i>NOBUTTON</i>	<i>BUTTON1</i>	<i>BUTTON2</i>	<i>BUTTON3</i>
-----------------	----------------	----------------	----------------

The *NOBUTTON* value indicates that no button was pressed or released. Java SE 6 added three methods to *MouseEvent* that obtain the coordinates of the mouse relative to the screen rather than the component. *Point getLocationOnScreen()*

- **The MouseWheelEvent Class:**

- ✓ The *MouseWheelEvent* class encapsulates a mouse wheel event.
- ✓ It is a subclass of *MouseEvent*. Not all mice have wheels. If a mouse has a wheel, it is located between the left and right buttons. Mouse wheels are used for scrolling.

MouseWheelEvent defines these two integer constants:

WHEEL_BLOCK_SCROLL	A page-up or page-down scroll event occurred.
WHEEL_UNIT_SCROLL	A line-up or line-down scroll event occurred.

Here is one of the constructors defined by *MouseWheelEvent*: *MouseWheelEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup, int scrollHow, int amount, int count)*

Here, *src* is a reference to the object that generated the event. The type of the event is specified by *type*. The system time at which the mouse event occurred is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when the event occurred. The coordinates of the mouse are passed in *x* and *y*.

- **The TextEvent Class:**

- ✓ Instances of this class describe text events.
- ✓ These are generated by *text fields* and *text areas* when characters are entered by a user or program. *TextEvent* defines the integer constant

TEXT_VALUE_CHANGED.

The one constructor for this class is shown here:

TextEvent(Object src, int type)

Here, *src* is a reference to the object that generated this event. The type of the event is specified by *type*. The *TextEvent* object does not include the characters currently in the text component that generated the event.

- **The WindowEvent Class:**

- ✓ There are ten types of window events.
- ✓ The *WindowEvent* class defines integer constants that can be used to identify them.

The constants and their meanings are shown in below table:

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

WindowEvent is a subclass of *ComponentEvent*. It defines several constructors. The first is *WindowEvent(Window src, int type)*

WindowEvent(Window src, int type, Window other)

WindowEvent(Window src, int type, int fromState, int toState)

WindowEvent(Window src, int type, Window other, int fromState, int toState)

❖ Handling Mouse Event:

- ✓ Handling mouse event using the delegation event model is actually quite easy.\

Just follow these two steps:

- ✓ Implement the appropriate interface in the listener so that it will receive the type of event desired.
 - ✓ Implement code to register and unregister (if necessary) the listener as a recipient for the event notifications.
- To handle mouse events, we must implement the *MouseListener* and the *MouseMotionListener* interfaces.
 - The following applet demonstrates the process.
 - It displays the current coordinates of the mouse in the applet's status window.
 - Each time a button is pressed, the word "Down" is displayed at the location of the mouse pointer.
 - Each time the button is released, the word "Up" is shown.
 - If a button is clicked, the message "Mouse clicked" is displayed in the upperleft corner of the applet display area.

// Demonstrate the mouse event handlers.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="MouseEvents" width=300 height=100>
</applet>
*/
public class MouseEvents extends Applet implements
MouseListener, MouseMotionListener {
    String msg = "";

    int mouseX = 0, mouseY = 0; // coordinates of mouse
    public void init() {
        addMouseListener(this);
        addMouseMotionListener(this);
    }
    // Handle mouse clicked.
    public void mouseClicked(MouseEvent me) {
        save coordinates mouseX = 0; mouseY = 10;
        msg = "Mouse clicked."; repaint();
    }
    // Handle mouse entered.
    public void mouseEntered(MouseEvent me) {
        save coordinates mouseX = 0; mouseY = 10;
        msg = "Mouse entered."; repaint();
    }
    // Handle mouse exited.
    public void mouseExited(MouseEvent me) {
        save coordinates mouseX = 0; mouseY = 10;
        msg = "Mouse exited."; repaint();
    }
}
```



```
// Handle button pressed.
public void mousePressed(MouseEvent me) {

    save coordinates mouseX = me.getX(); mouseY = me.getY(); msg = "Down";
    repaint();
}

// Handle button released.
public void mouseReleased(MouseEvent me) {
    save coordinates mouseX = me.getX(); mouseY = me.getY(); msg = "Up";
    repaint();
}

// Handle mouse dragged.
public void mouseDragged(MouseEvent me) {
    save coordinates mouseX = me.getX(); mouseY = me.getY(); msg = "**";
    showStatus("Dragging mouse at " + mouseX + ", " + mouseY);
    repaint();
}

// Handle mouse moved.
public void mouseMoved(MouseEvent me) { // show status
    showStatus("Moving mouse at " + me.getX() + ", " + me.getY());
}

Display msg in applet window at current X,Y location. public
void paint(Graphics g) {
    g.drawString(msg, mouseX, mouseY);
}
}
```

9.4. Swing:

- ✓ Swing is a set of program components for Java programmers that provide the ability to create graphical user interface (GUI) components, such as buttons and scroll bars, that are independent of the windowing system for specific operating system .
- ✓ Swing components are used with the Java Foundation Classes (JFC).
- ✓ Swing was created to address the limitations present in the AWT.
- ✓ It does this through two key features: ***lightweight components and a pluggable look and feel.***

I. Swing Components Are Lightweight:

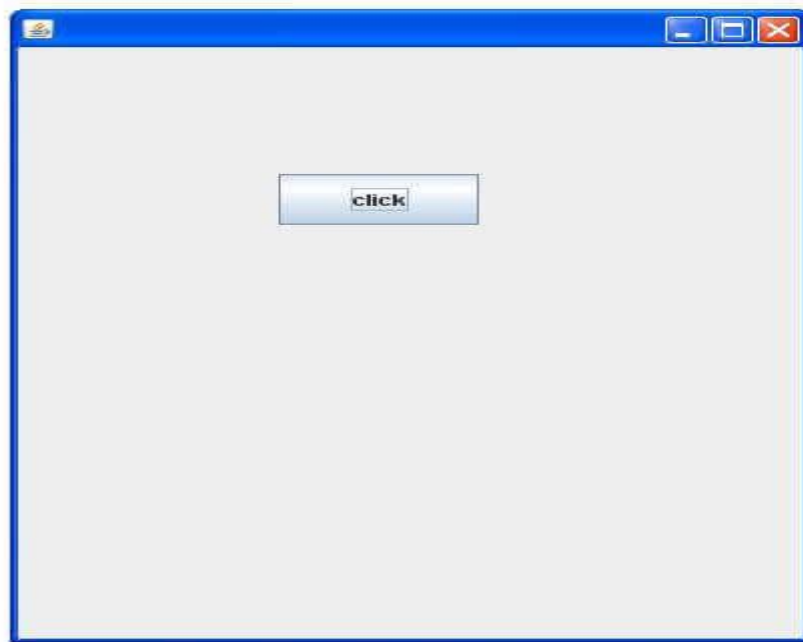
- With very few exceptions, Swing components are *lightweight*.
- This means that they are written entirely in Java and do *not map directly to platform-specific peers*.
- Because lightweight components are rendered using graphics primitives, they can be *transparent*, which enables nonrectangular shapes.
- Thus, lightweight components are more efficient and more flexible.
- Lightweight components do not translate into native peers, the *look and feel of each component is determined by Swing, not by the underlying operating system*.
- This means that each component will work in a consistent manner across all platforms.

II. Swing Supports a Pluggable Look and Feel:

- Swing component is *rendered by Java* code rather than by native peers, the look and feel of a component is under the control of Swing.
- This fact means that it is possible to separate the *look and feel* of a component from the *logic of the component*, and this is what Swing does.
- Separating out the look and feel provides a significant advantage: *it becomes possible to change the way that a component is rendered without affecting any of its other aspects*.
- Pluggable look-and-feels offer several important advantages.
- *It is possible to define a look and feel that is consistent across all platforms*. Conversely, it is possible to create a look and feel that acts like a specific platform.
- For example, if we know that an application will be running only in a Windows environment, it is possible to specify the Windows look and feel.
- It is also possible to design a custom look and feel. Finally, the look and feel can be changed dynamically at run time.

File: FirstSwingExample.java

```
import javax.swing.*;
public class FirstSwingExample {
    public static void main(String[] args) {
        JFrame f=new JFrame();//creating instance of JFrame
        JButton b=new JButton("click");//creating instance of JButton
        b.setBounds(130,100,100, 40);//x axis, y axis, width, height
        f.add(b);//adding button in JFrame
        f.setSize(400,500);//400 width and 500 height
        f.setLayout(null);//using no layout managers
        f.setVisible(true);//making the frame visible
    }
}
```

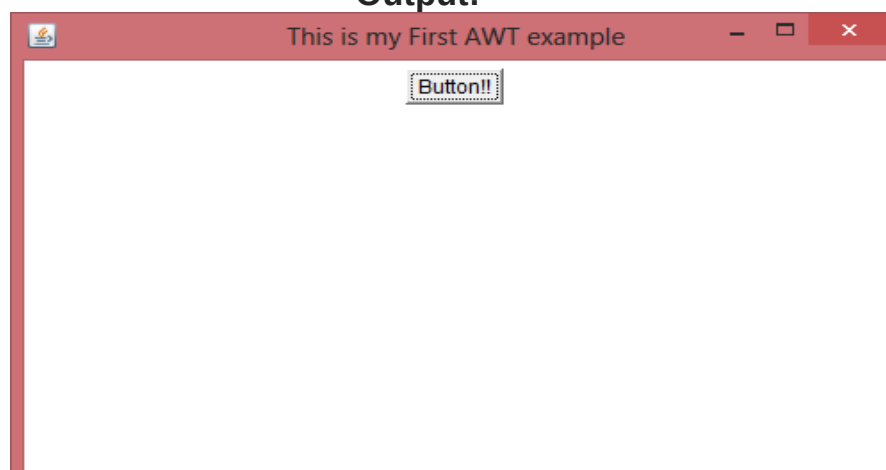


❖ AWT (Abstract Window Toolkit):

- ✓ Abstract Window Toolkit (AWT) is a set of application program interfaces (API's) used by Java programmers to create graphical user interface (GUI) objects, such as buttons, scroll bars, and windows.
- ✓ AWT is part of the Java Foundation Classes (JFC) from Sun Microsystems, the company that originated Java.

AWT Example 1: creating Frame by extending Frame class

```
import java.awt.*;
/* We have extended the Frame class here,
 * thus our class "SimpleExample" would behave
 * like a Frame*/
public class SimpleExample extends Frame{
    SimpleExample(){
        Button b=new Button("Button!!");
        // setting button position on screen
        b.setBounds(50,50,50,50);
        //adding button into frame
        add(b);
        //Setting Frame width and height
        setSize(500,300);
        //Setting the title of Frame
        setTitle("This is my First AWT example");
        //Setting the layout for the Frame
        setLayout(new FlowLayout());
        /* By default frame is not visible so
         * we are setting the visibility to true
         * to make it visible.
         */
        setVisible(true);
    }
    public static void main(String args[]){
        // Creating the instance of Frame
        SimpleExample fr=new SimpleExample();
    }
}
```

Output:

❖ Difference between AWT and Swing:

No.	Java AWT	Java Swing
1)	AWT components are platform-dependent.	Java swing components are platform-independent.
2)	AWT components are heavyweight.	Swing components are lightweight.
3)	AWT doesn't support pluggable look and feel.	Swing supports pluggable look and feel.
4)	AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scroll panes, color chooser, tabbed pane etc.
5)	AWT doesn't follows MVC (Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC.

9.5. Separating GUI and Application Code (MVC Pattern):

- The Swing designers turned to a well-known design pattern: the **model-view controller** pattern.
- This pattern, like many other design patterns, goes back to one of the principles of object-oriented design.
- Don't make one object responsible for too much.
- Don't have a single button class do everything.
- Instead, have the look and feel of the component associated with one object and store the contents in another object.
- The model-view-controller (MVC) design pattern teaches how to accomplish this.

Implement three separate classes:

- The model, which stores the contents;
 - The view, which displays the contents;
 - The controller, which handles user input.
- ✓ The *model* stores the contents and has no user interface.
 - ✓ For a button, the content is pretty trivial-it is just a small set of flags that tells whether the button is currently pushed in or out, whether it is active or inactive, and so on. The model must implement methods to change the contents and to discover what the contents are. One of the advantages of the model-view-controller pattern is that a *model* can have multiple *views*, each showing a different part or aspect of the full contents. When the model is updated through the *controller* of one of the views, it tells both attached *views* about the change. When the *views* are notified, they refresh themselves automatically.
 - ✓ The *controller* handles the user input events such as mouse clicks and keystrokes.
 - ✓ It then decides whether to translate these events into changes in the *model* or the *view*.

- ✓ For example, if the user presses a character key in a text box, the controller calls the “insert character” command of the model.
- ✓ *The model then tells the view to update itself.* The view never knows why the text changed. But if the user presses a key, then the controller may tell the view to scroll. Scrolling the view has no effect on the underlying text, so the model never knows that this event happened.

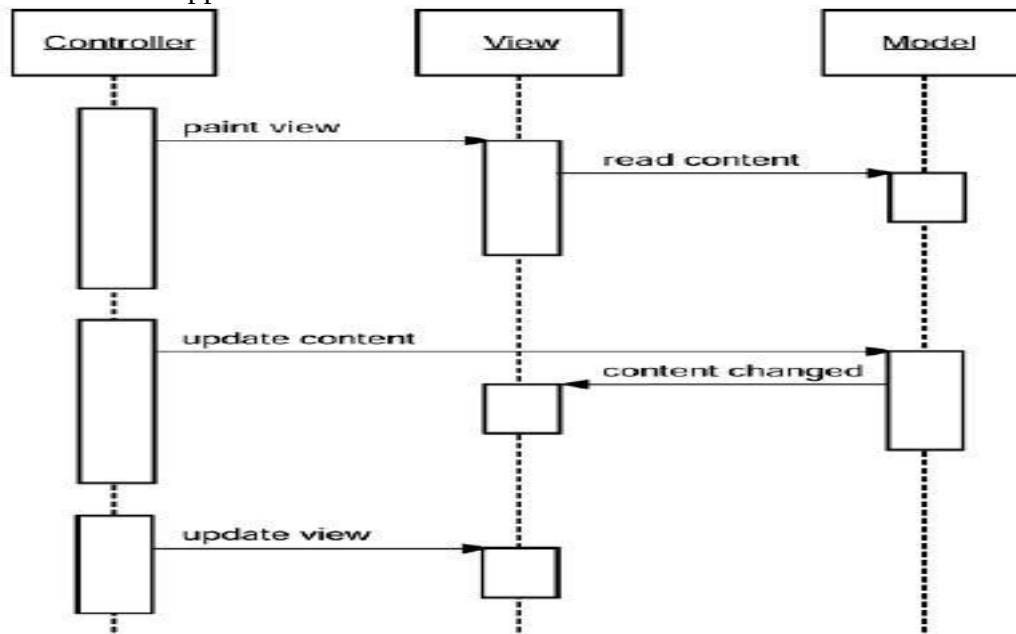


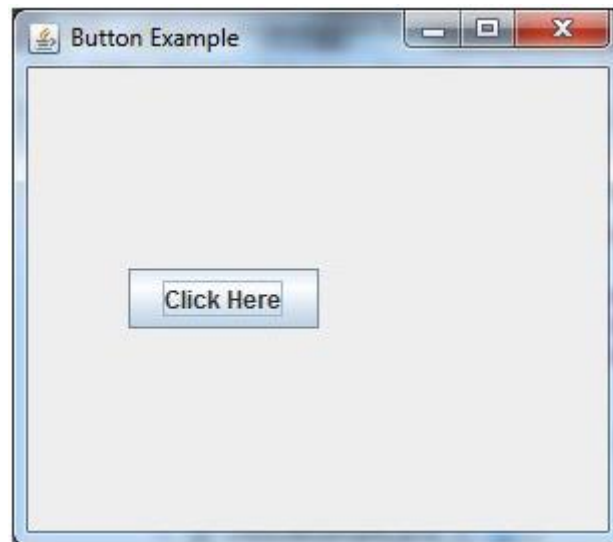
Fig: Interaction between Model, View and Controller objects

- ✓ Besides being “*the right thing to do*,” the **model-view-controller** pattern was attractive for the Swing designers because it allowed them to implement pluggable look and feel.
- ✓ The model of a button or text field is independent of the look-and-feel.

❖ Building GUI with Swing:

```

import javax.swing.*;
public class ButtonExample {
    public static void main(String[] args) {
        JFrame f=new JFrame("Button Example");
        JButton b=new JButton("Click Here");
        b.setBounds(50,100,95,30);
        f.add(b);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
}
Output:
  
```

**Important Questions from this unit:**

- Q.1) How swing handle the event in java? Demonstrate using example.
- Q.2) Differentiate between AWT and Swing?
- Q.3) Write a java program to print hello world while clicking button using swing.
- Q.4) what is event Delegation model? Write a program to change the background color using button.
- Q.5) Write a code snippet for jButton, JLabel, JTextArea, JCheckBox, JRadioButton, JList, JTable, JPasswordField.
- Q.6) what is swing? How GUI is implemented using swing.
- Q.7) Explain about mouse event handling with code snippet.

Changing background color using button:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class ButtonTest
{
    public static void main(String[] args)
    {
        ButtonFrame frame = new ButtonFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); frame.show();
    }
}
/**
A frame with a button panel */
class ButtonFrame extends JFrame
{
    public ButtonFrame()
    {
        setTitle("ButtonTest");

        setSize(WIDTH, HEIGHT);

        // add panel to frame
```

```
ButtonPanel panel = new ButtonPanel(); Container contentPane = getContentPane();
contentPane.add(panel);
}
public static final int WIDTH = 300;
public static final int HEIGHT = 200;
}
/** A panel with three buttons.      */

class ButtonPanel extends JPanel
{
public ButtonPanel()
{
// create buttons
JButton yellowButton = new JButton("Yellow");
JButton blueButton = new JButton("Blue");
JButton redButton = new JButton("Red");
// add buttons to panel
add(yellowButton);
add(blueButton);
add(redButton);
// create button actions
ColorAction yellowAction = new ColorAction(Color.yellow); ColorAction blueAction = new
ColorAction(Color.blue);
ColorAction redAction = new ColorAction(Color.red); // associate actions with buttons
yellowButton.addActionListener(yellowAction);
blueButton.addActionListener(blueAction);
redButton.addActionListener(redAction);
}

/** An action listener that sets the panel's background color. */ private class ColorAction
implements ActionListener
{
public ColorAction(Color c)
{
backgroundColor = c;
}
public void actionPerformed(ActionEvent event)
{
setBackground(backgroundColor);
repaint();
}
private color backgroundColor;
}
}
```